



D0 · DELIVERABLE

Submission Index

HireSense: deliverables, outcomes and rubric mapping for Case Study 31.

Start here. Six documents answer the six required deliverables, backed by a working prototype whose data, tests and code generate every number in them.

Contents

1	Deliverables required by the case study	2	Outcomes the case study asks for
3	Mapping to the course rubric	4	The working prototype

AUTHOR	COURSE	DOCUMENT
Anshuman Atrey	Software Engineering & Project Management	D0 · v1.0
Roll 150096724029	B.Tech CSE · Batch 2024–28 · Semester V	1 October 2026 · ITM Skills University

1 Deliverables required by the case study

CASE-STUDY DELIVERABLE	DOCUMENT	WHAT IT CONTAINS
White-Box Test Report: CFG, cyclomatic complexity, independent paths, coverage evidence	D1	CFG (11 nodes, 14 edges), $V(G) = 5$ by three methods, 8/8 paths tested, boundary/condition/metamorphic tests, the silent-parse-failure test, coverage, 11 bug reports
SRS: explainability, configurability, logging and fairness requirements with thresholds	D2	IEEE 830 structure, stakeholders, 22 measurable functional requirements, NFRs, use cases, traceability matrix to code and tests, feasibility
Design Pack: class diagram, activity diagram, module cohesion justification	D3	Architecture, cohesion/coupling with a measured dependency graph, class, activity, sequence, state, deployment and ER diagrams, UI screens
Audit Specification: metrics, frequency, thresholds, breach response procedure	D4	Selection rate, impact ratio, z-test, 9 acceptance tests, rolling windows, severity ladder, RACI procedure, live results
Project Plan: incremental WBS, Gantt, risk register including reputational risk	D5	Charter, 7 strangler increments, WBS, work package WP-P, COCOMO, PERT/CPM, Gantt, sprints, backlog, burndown, 12-risk register
Maintenance Process: recurring audit cycle as preventive maintenance	D6	Maintenance types, drift sources, monthly cycle and calendar, change and release gates, KPIs, audit history, maintenance log

2 Outcomes the case study asks for

EXPECTED OUTCOME	EVIDENCE
A cyclomatic complexity value with the complete independent path set and tests	D1 §3–4: $V(G) = 5$; basis paths P1–P5 plus P6–P8; every path has a passing test
A test that specifically catches silent parse failure	D1 §6: scanned résumé → legacy rejects silently; HireSense routes to review (and OCR recovers it)
Fairness expressed as a number with a defined consequence when breached	D4 §4–7: $IR \geq 0.80$ for groups with $n \geq 30$; a significant breach pauses automated rejection
A modular design where changing a ranking rule does not touch parsing	D3 §2: ranking and scoring import nothing from parsing (measured); criteria are versioned data
A recurring audit process rather than a one-off report	D4 §5 + D6 §3: monthly rolling audits, triggered audits, governance review, KPIs

3 Mapping to the course rubric

RUBRIC ITEM (MARKS)	WHERE
Problem analysis & requirement gathering (2)	D2 §1–3
SRS & documentation quality (3)	D2 (IEEE 830), D0–D6
UML design & architecture (3)	D3: use-case (D2 §6), class, activity, sequence, state, deployment, ER
Use of software-engineering tools (3)	Bun test + coverage, JUnit, Mermaid UML, SQLite, Git, Next.js/shadcn, Ollama, PP-OCR, headless-browser doc build
Project planning: WBS, Gantt, agile artefacts (3)	D5 §3–7
Testing artefacts: test cases, bug reports (2)	D1 §4–9

RUBRIC ITEM (MARKS)	WHERE
GitHub repository & version control (2)	hiresense/ monorepo (D5 §9)
Presentation & viva (2)	Live prototype demo

4 The working prototype

APPLICATIONS SCREENED

2,000

synthetic, 6 roles, 12 months

PARSE SUCCESS

99.0%

legacy 71.0%

SILENT FAILURES

580 → 6

legacy → HireSense

AUTOMATED TESTS

82 ✓

165 assertions, 0 failures

Every number in D0–D6 is generated from the running prototype by docs/build.ts : the seeded database, a fresh test run with coverage, and the code itself. The documents cannot drift from the system.

```
cd hiresense
bun run setup      # install + seed 2,000 synthetic résumés (PDFs, parses, decisions, audits)
bun run dev        # API http://localhost:8787/api · web http://localhost:3000
bun run test       # unit + integration tests with coverage
cd docs && bun run build  # regenerate these PDFs from live data
```

Stack: Next.js 16 · shadcn/ui · Bun + Hono · bun:sqlite · pdf.js · PP-OCRv6 tiny (ONNX) · optional Qwen3 1.7B via Ollama. All candidate data is synthetic. [Case-study figures used as given: routine, 0.6, 29%, 80%](#)